

Jakub

Osoba 4 - Rdzen silnika, integracja, logika gry, HUD

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/Engine.*, main.cpp, CMakeLists.txt, Demo/Gameplay.cpp, Demo/ShootingGallery.h, Demo/Constants.h

0. Gdzie sa moje pliki (mapa do prezentacji)

Strzałka <<< pokazuje pliki, które robiłem ja (Jakub).

```
pgk/
|
+-- main.cpp          <<< MOJE  (start programu, spina wszystko)
+-- CMakeLists.txt    <<< MOJE  (konfiguracja kompilacji)
+-- README.md Doxyfile .gitignore <<< MOJE  (infrastruktura)
|
+-- Engine/
|   +-- Math3D.* SceneNode.*          (Lukasz)
|   +-- PrimitiveNode.*              (Rogert)
|   +-- Camera.* Light.* Texture.*    (Kacper)
|   +-- Engine.h / Engine.cpp <<< MOJE  (okno, petla, wejscie, callbacki)
|
+-- Demo/
|   +-- Room.cpp Obstacles.cpp Targets.cpp (Lukasz/Rogert/Kacper)
|   +-- ShootingGallery.h <<< MOJE  (deklaracja calej klasy gry)
|   +-- Constants.h <<< MOJE  (stale gry - balans)
|   +-- Gameplay.cpp <<< MOJE  (logika gry, strzelanie, HUD)
```

W skrocie: ja jestem 'spinaczem'. Robie **rdzen silnika** (okno przez FreeGLUT, petla rysowania, obsługa klawiatury i myszy) oraz **glowna logike gry** (co sie dzieje co klatke, strzelanie, punkty, zycia, HUD). Naglowek ShootingGallery.h jest 'kontraktem' - reszta zespolu implementuje swoje funkcje, które ja tu zadeklarowalem.

1. O co chodzi w moich plikach (po ludzku)

Engine to silnik bez gry - sam w sobie tylko otwiera okno, ustawia OpenGL i kreci petla: 'policz -> narysuj -> powtorz'. Gra podpinia sie do niego przez **callbacki** (funkcje zwrotne): silnik mowi 'mamy nowa klatke' i wola moja funkcje gry.

Gameplay.cpp to mozg gry: porusza graczem, sprawdza strzaly, liczy punkty i zycia, rysuje HUD (celownik, wyniki, ekran konca gry). **Constants.h** trzyma wszystkie liczby balansu w jednym miejscu (np. ile zyc, jak dlugo zyje cel).

2. Engine - serce silnika

2.1. Petla czasu (zmienne FPS)

FreeGLUT wola moja funkcje onTimer co pewien czas. Licze **dt** (czas od ostatniej klatki) i przekazuje go do gry - dzieki temu ruch jest plynny niezaleznie od FPS.

Engine.cpp - petla czasu z dt i pauza

```
void Engine::onTimer() {
    int now = glutGet(GLUT_ELAPSED_TIME);
    if (paused) { lastTickMs = now; glutPostRedisplay();
        glutTimerFunc(1000/targetFPS, timerCallback, 0); return; }
    float dt = (now - lastTickMs) / 1000.0f; // sekundy od ostatniej klatki
    lastTickMs = now;
    ... // ruch kamery silnika (gdy tryb ENGINE_ORBIT)
    if (updateCallback) updateCallback(dt); // <- logika gry
    glutPostRedisplay(); // popros o przerysowanie
    glutTimerFunc(1000/targetFPS, timerCallback, 0); // zaplanuj kolejna klatke
}
```

Klawisze +/- zmieniaja targetFPS, czyli jak czesto planujemy klatke - to spelnia wymaganie 'zmienna szybkoosc odswiezania'. Pauza (ESC) pomija logike gry, ale dalej rysuje (zeby pokazac ekran PAUZA).

2.2. Obsluga wejscia + free mouse look

Klawiatura: trzymam tablice keys[256] (true gdy wcisniety). Gra pyta isKeyDown('w'). Mysz w trybie FPP chowam kursor i po kazdym ruchu przesuwam go z powrotem na srodek ekranu - dzieki temu mozna sie obracac w nieskonczonosc.

Engine.cpp - rozgladanie sie myszka (uproszczone)

```
void Engine::handleMouseMotion(int x, int y) {
    if (paused) return;
    if (freeMouseLook) {
        int cx = windowWidth/2, cy = windowHeight/2;
        if (x==cx && y==cy) return; // ignoruj wlasny warp
        cameraYaw += (x-cx) * mouseSensitivity;
        cameraPitch += -(y-cy) * mouseSensitivity;
        cameraPitch = clamp(cameraPitch, -85deg, +85deg);
        glutWarpPointer(cx, cy); // wroc kursor na srodek
    }
}
```

2.3. Callbacki - jak silnik laczy sie z gra

Silnik nie wie nic o 'strzelnicy'. Trzyma tylko wskazniki na funkcje (std::function), ktore gra mu daje. To rozdziela silnik od konkretnej gry.

main.cpp - podpiecie logiki gry pod silnik

```
// w main.cpp - gra rejestruje sie w silniku:  
engine.setUpdateCallback([&](float dt){ game.onUpdate(dt); });  
engine.setShootCallback ([&]() { game.onShoot(); });  
engine.setHUDCallback    ([&]() { game.onHUD(); });  
engine.setResetCallback ([&]() { game.onReset(); });
```

2.4. Dlaczego callbacki GLUT sa statyczne

FreeGLUT to biblioteka w C - przyjmuje zwykle funkcje (wskazniki), nie metody klasy. Dlatego mam statyczne funkcje, ktore przez wskaznik 'instance' wolaja prawdziwe metody obiektu Engine.

Engine.cpp - mostek miedzy C-owym GLUT a obiektem C++

```
static Engine* instance; // jedyny obiekt silnika  
  
void Engine::displayCallback() { if (instance) instance->render(); }  
void Engine::timerCallback(int){ if (instance) instance->onTimer(); }  
// GLUT dostaje displayCallback, a ta woła instance->render()
```

3. Demo/Gameplay.cpp - logika gry

Tu jest konstruktor gry (ustawia swiatla, wola buildRoom/buildObstacles itd. od kolegow, konfiguruje kamere FPP) oraz 4 glowne funkcje: onUpdate, onShoot, onHUD, onReset.

3.1. onShoot - strzelanie (raycast)

Gameplay.cpp - rdzen strzelania

```
void ShootingGallery::onShoot() {
    if (gameOver_ || targets_.empty()) return;
    // odswiez kamere NAJNOWSZYM yaw/pitch (mysz mogla sie ruszyc!)
    engine_.getCamera()->setFirstPerson(playerEye_,
        engine_.getCameraYaw(), engine_.getCameraPitch());
    Vec3 origin = engine_.getCamera()->eyePosition();
    Vec3 dir    = engine_.getCamera()->lookDirection();

    float closestT = MAX_SHOOT_DIST; int hitIdx = -1;
    for (size_t i=0; i<targets_.size(); ++i) {
        float t;
        if (raySphereIntersect(origin, dir, currentTargetPos(targets_[i]),
            TARGET_RADIUS*HIT_TOLERANCE, t)
            && t > 0 && t < closestT) { closestT = t; hitIdx = i; }
    }
    if (hitIdx >= 0 && obstacleBlocksRay(origin, dir, closestT)) hitIdx = -1;
    // hitIdx >= 0 -> trafienie; inaczej pudlo (-1 zycie)
}
```

Najwazniejszy szczegol: przed raycastem odswiezam kamere najnowszym yaw/pitch. To naprawilo bug, gdzie strzaly mijaly cel - mysz mogla sie ruszyc miedzy klatka a wcisnieciem spacji, wiec kierunek byl nieaktualny. **HIT_TOLERANCE** (1.4) lekko powieksza cel, zeby latwiej bylo trafic.

3.2. onUpdate - co klatke

Porusza graczem (WASD), wola kolizje (Rogert), aktualizuje kamere, odlicza timery efektow, sprawdza czy cele nie wygasly, respawnuje fale. Wszystko skalowane przez **dt**, zeby tempo bylo niezalezne od FPS.

Gameplay.cpp - ruch gracza z dt

```
float speed = PLAYER_MOVE_SPEED * dt;          // metry = predkosc * czas
if (engine_.isKeyDown('w')) playerEye_ += fwd * speed;
...
applyObstacleCollision();                      // wypchnij z przeszkod
playerEye_.x = clamp(playerEye_.x, sciany...); // nie wychodz z pokoju
```

3.3. onHUD - interfejs 2D

Rysuje na plasko (2D): celownik na srodku, wyniki/zycia/czas w rogu, pasek czasu celu, czerwony/zielony blysk po pudle/trafieniu, ekran Game Over i Pauza. Uzywam pomocnikow drawString z silnika.

4. Constants.h i ShootingGallery.h

Constants.h to wszystkie liczby balansu w jednym miejscu (namespace sg): rozmiar pokoju, ile zyc, jak dlugo zyje cel, predkosc gracza. Zmiana trudnosci = zmiana jednej liczby. Uzywam constexpr, ktore w C++17 sa inline - mozna wlaczac naglowek w wielu plikach bez bledu linkera.

ShootingGallery.h deklaruje cala klase gry: struktury (Target, Obstacle...), wszystkie pola i metody. To 'kontrakt' - ja go pisze, a koleczy implementuja swoje metody (buildRoom, buildObstacles, spawnWave) w swoich plikach .cpp. Dzieki temu mozemy pracowac rownolegle bez kolizji w gicie.

5. CMakeLists.txt - jak to sie kompiluje

CMake zbiera wszystkie pliki .cpp, znajduje OpenGL i FreeGLUT (rozne sciezki dla Windows/macOS/Linux) i tworzy plik wykonywalny. Dodatkowo kopiuje folder assets (tekstury BMP) i freeglut.dll obok exe, zeby gra je znalazla.

CMakeLists.txt - szkielet

```
set(ENGINE_SOURCES Engine/Engine.cpp Engine/Math3D.cpp ...)  
set(DEMO_SOURCES   Demo/Room.cpp Demo/Obstacles.cpp  
                  Demo/Targets.cpp Demo/Gameplay.cpp)  
add_executable(${PROJECT_NAME} main.cpp ${ENGINE_SOURCES} ${DEMO_SOURCES})  
# osobny target kopiuje assets/ przy KAZDYM buildzie (ALL)  
add_custom_target(copy_assets ALL COMMAND ... copy_directory assets ...)
```

6. Pytania od wykładowcy - przygotowanie

Engine - petla i czas

P: Jak działa petla główna w waszym silniku?

O: FreeGLUT po glutMainLoop wola zarejestrowane callbacki. timerCallback (onTimer) odpala się co $1000/\text{targetFPS}$ ms - tam licze Δt , aktualizuje logikę i wołam glutPostRedisplay, które prosi o przerysowanie (displayCallback -> render). Potem planuje kolejny timer. Tak kręci się petla 'licz -> rysuj'.

P: Co to jest Δt i dlaczego mnożysz przez nie ruch?

O: Δt to czas od poprzedniej klatki w sekundach. Mnożąc prędkość przez Δt , dostaję faktyczny dystans (droga = prędkość * czas). Dzięki temu przy 30 i 120 FPS gracz porusza się tak samo szybko - inaczej przy wyższym FPS ruszałby się szybciej.

P: Jak realizujecie zmienną szybkość odświeżania?

O: targetFPS steruje interwałem timera ($1000/\text{targetFPS}$ ms). Klawisze +/- zmieniają go o 10. Pokazuje go na HUD obok zmierzonego FPS. To pokazuje, że petla jest sterowalna.

P: Jak działa pauza?

O: Flaga paused. W onTimer, gdy paused, pomijam logikę gry (nie wołam updateCallback), ale dalej rysuję (żeby pokazać napis PAUZA). Aktualizuję lastTickMs, żeby po wznowieniu Δt nie było ogromne. ESC przełącza pauzę, Q w pauzie wychodzi.

Engine - wejście i callbacki

P: Dlaczego callbacki GLUT są statyczne?

O: FreeGLUT jest biblioteka C - przyjmuje wskaźniki na zwykłe funkcje, nie metody klasy (te mają ukryty wskaźnik this). Dlatego robię statyczne funkcje, które przez globalny wskaźnik 'instance' wołają prawdziwe metody obiektu Engine. To typowy mostek C/C++.

P: Jak działa free mouse look?

O: Chowam kursor i po każdym ruchu liczę przesunięcie względem środka ekranu, dodaje do yaw/pitch, a potem przesuwam kursor z powrotem na środek (glutWarpPointer). Dzięki temu kursor nigdy nie dochodzi do krawędzi i można obracać się bez końca.

P: Po co callbacki (std::function) zamiast wpisać logikę gry wprost w silnik?

O: Żeby silnik był niezależny od konkretnej gry. Engine wie tylko 'jest nowa klatka' i woła podaną funkcję. Można podpiąć dowolną grę bez zmiany silnika. To luźne sprzężenie - dobra praktyka projektowa.

P: Co to jest CameraControlMode?

O: Przełącznik kto rządzi kamerą. ENGINE_ORBIT - silnik sam obraca kamerą (tryb testowy). GAME_CONTROLLED - silnik nie rusza kamery, robi to gra w swoim onUpdate (FPP gracza). Gra ustawia GAME_CONTROLLED.

Gameplay - logika i strzelanie

P: Wytłumacz krok po kroku co dzieje się przy strzale.

O: 1) Odświeżam kamerę najnowszym yaw/pitch. 2) Biorę origin (oko) i dir (kierunek patrzenia). 3) Dla każdego celu robię raySphereIntersect i szukam najbliższego trafionego (najmniejsze t). 4) Sprawdzam, czy żadna przeszkoda nie jest bliżej (obstacleBlocksRay) - jak tak, to pudło. 5) Trafienie: +1 punkt, znika cel, bonus +2s dla reszty fali. Pudło: -1 życie.

P: Dlaczego odświeżasz kamerę przed raycastem - to był wasz bug?

O: Tak. Kamera (yaw/pitch) była aktualizowana raz na klatkę w onUpdate. Ale gracz mógł ruszyć mysz i wcisnąć spację MIĘDZY klatkami - wtedy raycast szedł w stary kierunek, mimo że celownik wskazywał nowy. Strzały mijaly cel. Naprawa: tuż przed raycastem ustawiam kamerę aktualnymi klatkami.

P: Co to jest HIT_TOLERANCE?

O: Mnożnik (1.4), którym powiększam promień celu tylko na potrzeby trafienia. Dzięki temu można celować trochę obok środka i nadal trafić - gra jest mniej frustrująca. Wizualnie cel zostaje tej samej wielkości.

P: Jak działa bonus +2s po trafieniu?

O: Gdy w fali zostają jeszcze cele, do każdego dodaje 2 sekundy życia - przez przesunięcie ich spawnTime do przodu. Timeout liczy się jako totalTime - spawnTime, więc przesunięcie spawnTime odracza wygasnięcie. Prosty trik bez dodatkowego pola.

Constants / Header / CMake

P: Dlaczego stale są constexpr w nagłówku, a nie #define?

O: constexpr ma typ i zasięg (namespace sg), jest bezpieczniejsze niż #define (zwykła podmiana tekstu). W C++17 constexpr w namespace jest niejawnie inline, więc nagłówek można włączać w wielu plikach .cpp bez błędów wielokrotnej definicji.

P: Po co rozdzielacie .h (deklaracja) od .cpp (implementacja)?

O: Nagłówek to 'kontrakt' - mówi jakie są klasy i metody. Implementacja jest osobno. Dzięki temu 4 osoby mogą pisać różne pliki .cpp tej samej klasy równolegle, nie wchodząc sobie w drogę, a kompilator wie z nagłówka, co istnieje.

P: Dlaczego assets kopiujecie osobnym targetem z ALL, a nie POST_BUILD?

O: POST_BUILD odpala sie tylko gdy exe jest przebudowywane. Jak dodam nowy plik BMP, ale kod sie nie zmienil, exe nie jest linkowane i kopiowanie sie nie uruchamia. Osobny target z ALL wykonuje sie przy kazdym buildzie, wiec nowe tekstury zawsze trafiaja obok exe.

P: Jak CMake radzi sobie z roznymi systemami?

O: Sprawdzam zmienne APPLE / WIN32. Na Windows uzywam vcpkg lub MSYS2 (ucrt64), na macOS frameworka OpenGL i Homebrew, na Linux find_package(GLUT). Dzieki temu ten sam projekt kompiluje sie na 3 systemach.